Project Report: Health and Fitness Club Management System

Course: COMP 3005 - Fall 2025 **Title:** Health and Fitness Club Management System **Team Members:** Soliman Elkhouli, Yousuf Elshahry **Date:** 30-11-2025

---

1. Conceptual Database Design (ER Model)

The core objective of this project was to design and implement a robust, database-driven platform for managing a modern fitness center. Our Entity-Relationship (ER) Model was designed to be comprehensive, supporting all functional requirements across the three user roles (Member, Trainer, Admin).

Component	**Implemented**
Entities	12
Relationships	12+ (Foreign Keys)

The 12 primary entities are: `Member`, `Trainer`, `Admin`, `Room`, `Equipment`, `FitnessGoal`, `HealthMetric`, `PersonalTrainingSession`, `GroupClass`, `ClassEnrollment`, `TrainerAvailability`, and `Billing`.

The model is characterized by the following key design decisions:

- **Historical Data Tracking:** The `HealthMetric` entity is designed with a `TIMESTAMP` to record multiple entries for a member's weight, heart rate, etc., ensuring previous data is never overwritten and allowing for trend analysis.
- **Many-to-Many Relationships:** The relationship between `Member` and `GroupClass` is resolved using the junction table `ClassEnrollment`.
- **Role Separation:** Separate tables (`Member`, `Trainer`, `Admin`) are used to enforce distinct access privileges and functional responsibilities, as required by the problem statement.

2. Mapping ER to Relational Schema

The conceptual ER Model was directly translated into a relational schema using PostgreSQL's Data Definition Language (DDL) in the `DDL.sql` file. This process ensured a high degree of data integrity and consistency.

Key Mapping Elements:

Relational Element	Implementation Detail	Purpose
Primary Keys	`SERIAL PRIMARY KEY`	Ensures a unique, non-null identifier for every record (e.g., `member_id`, `trainer_id`).
Foreign Keys	`FOREIGN KEY (...) REFERENCES ... ON DELETE CASCADE`	Enforces referential integrity. For example, deleting a `Member` automatically cascades and deletes their associated `FitnessGoal` and `HealthMetric` records.
Data Validation	`CHECK` Constraints	Used to enforce business rules at the database level, such as ensuring `capacity > 0` for `Room` and `GroupClass`, and validating that `end_time > start_time` for all scheduled sessions.
Uniqueness	`UNIQUE` Constraints	Applied to critical fields like `email` in the `Member`, `Trainer`, and `Admin` tables to prevent duplicate user accounts.

3. Schema Quality and Normalization

The final relational schema is normalized to at least the **Third Normal Form (3NF)**.

Normalization Strategy:

1. **First Normal Form (1NF):** Achieved by ensuring all attributes are atomic and eliminating repeating groups. For instance, each `HealthMetric` record contains a single set of measurements at a specific timestamp.
2. **Second Normal Form (2NF):** Achieved by ensuring all non-key attributes are fully dependent on the primary key. For example, member-specific details are stored only in the `Member` table, not redundantly in the `FitnessGoal` table.
3. **Third Normal Form (3NF):** Achieved by eliminating transitive dependencies. For example, room details are stored only in the `Room` table, and other tables reference rooms only via the `room_id` foreign key, preventing redundancy and update anomalies.

This high level of normalization minimizes data redundancy, improves data consistency, and simplifies maintenance and query logic.

4. SQL Code (DDL, DML, Queries)

The project utilizes PostgreSQL and demonstrates proficiency in advanced SQL features, as required.

Advanced SQL Features Implemented:

Feature	**Name**	**Purpose and Implementation**
View	`MemberDashboard`	A complex query defined in `DDL.sql` that aggregates a member's latest health metrics, active goals, and upcoming sessions into a single, easily queryable virtual table for the member dashboard function.
Trigger 1	`trg_check_room_availability_pt`	A `BEFORE INSERT OR UPDATE` trigger on the `PersonalTrainingSession` table. It executes a function that checks for time and room conflicts against both `PersonalTrainingSessio`

Feature	Name	Purpose and Implementation
		`n` and `GroupClass` tables, preventing double-booking.
Trigger 2	`trg_check_class_capacity`	A `BEFORE INSERT` trigger on the `ClassEnrollment` table. It checks the current enrollment count against the class's maximum capacity, raising an exception if the class is full.
Indexes	5 Indexes (e.g., `idx_health_metric_date`)	Created on frequently searched columns (`email`, `session_date`, `recorded_date`) to significantly improve query performance, especially for time-series data and user lookups.

The `DML.sql` file contains sufficient sample data (at least 3-5 records per table) to fully test all implemented application functions and demonstrate the system's operational workflow.

5. Application Functionality

The application was developed in Python, using the `psycopg2` library for direct interaction with the PostgreSQL database. The implemented functions are separated into three modules: `member_operations.py`, `trainer_operations.py`, and `admin_operations.py`.

The minimum requirement for a team of two was **10 total operations**. Our team implemented **13 distinct operations**, covering every possible function listed in the project requirements.

User Role	Implemented Functions (13 Total)	Fulfillment Status
Member (6/4)	User Registration, Profile Management, Health History, Dashboard (uses View), PT Session Scheduling (uses	**Exceeded**

User Role	Implemented Functions (13 Total)	Fulfillment Status
	Trigger), Group Class Registration (uses Trigger)	
Trainer (3/2)	Set Availability, Schedule View, Member Lookup (read-only)	**Exceeded**
Admin (4/2)	Room Booking (uses Trigger), Equipment Maintenance, Class Management, Billing & Payment (simulated)	**Exceeded**

Each function is backed by specific SQL logic, ensuring that all data manipulation adheres to the defined schema and constraints. For example, the **PT Session Scheduling** function first checks the `TrainerAvailability` table and then attempts the insertion, relying on the `trg_check_room_availability_pt` trigger to handle the final conflict validation.

6. User Interface / CLI Flow

The application uses a **Command Line Interface (CLI)** built in Python. While simple, the interface is functional, clear, and enforces proper role separation.

User Flow:

1. Upon starting the application (`python main.py`), the user is presented with a main menu to select their role: Member, Trainer, or Admin.
2. Selecting a role directs the user to a dedicated sub-menu containing only the operations relevant to that role (e.g., the Trainer menu does not contain Billing or Registration options).
3. All user input is validated before being passed to the database, and the application provides clear, descriptive feedback messages for both successful operations and constraint violations (e.g., when a class is full or a room is double-booked).

This role-based menu structure ensures that the system maintains the required separation of duties and prevents unauthorized access to sensitive operations.

---

7. Team Contribution and Work Split

The project implementation and demonstration were divided equally between the two team members to ensure full participation and coverage of all required components.

Team Member	**Primary Implementation Focus**	**Primary Video Demonstration Focus**
Yousuf Elshahry	Database Design (ERD, DDL, DML, Advanced SQL Features)	Database Design, Trainer Operations (Set Availability, Schedule View, Member Lookup)
Soliman Elkhouli	Application Logic (CLI, User Flow, Database Connection)	Member Operations (Registration, Health History, Scheduling), Admin Operations (Room Booking, Equipment, Billing)

Video Demonstration

The video demonstration, available at https://www.youtube.com/watch?v=lyJYgCf9qsc